

```
③         return WizCoin(other.galleons + self.galleons, other.sickles +
self.sickles, other.knuts + self.knuts)
```

When a `WizCoin` object is on the left side of the `+` operator, Python calls the `__add__()` method ❶ and passes in the value on the right side of the `+` operator for the `other` parameter. (The parameter can be named anything, but `other` is the convention.)

Keep in mind that you can pass any type of object to the `__add__()` method, so the method must include type checks ❷. For example, it doesn't make sense to add an integer or a float to a `WizCoin` object, because we don't know whether it should be added to the `galleons`, `sickles`, or `knuts` amount.

The `__add__()` method creates a new `WizCoin` object with amounts equal to the sum of the `galleons`, `sickles`, and `knuts` attributes of `self` and `other` ❸. Because these three attributes contain integers, we can use the `+` operator on them. Now that we've overloaded the `+` operator for the `WizCoin` class, we can use the `+` operator on `WizCoin` objects.

Overloading the `+` operator like this allows us to write more readable code. For example, enter the following into the interactive shell:

```
>>> import wizcoin
>>> purse = wizcoin.WizCoin(2, 5, 10) # Create a WizCoin object.
>>> tipJar = wizcoin.WizCoin(0, 0, 37) # Create another WizCoin object.
>>> purse + tipJar # Creates a new WizCoin object with the sum amount.
WizCoin(2, 5, 47)
```

If the wrong type of object is passed for `other`, the dunder method shouldn't raise an exception but rather return the built-in value `NotImplemented`. For example, in the following code, `other` is an integer:

```
>>> import wizcoin
>>> purse = wizcoin.WizCoin(2, 5, 10)
>>> purse + 42 # WizCoin objects and integers can't be added together.
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'WizCoin' and 'int'
```

Returning `NotImplemented` signals Python to try calling other methods to perform this operation. (See “Reflected Numeric Dunder Methods” later in